

установка и настройка jenkins

🕒 Created	August 28, 2026 3:21 PM
🏷️ Tags	devops practices

На всех ВМ выполните:

```
sudo apt update && sudo apt full-upgrade -y  
sudo apt install -y curl wget gnupg git unzip
```

На control-node создайте пару SSH-ключей и скопируйте публичный ключ на целевые серверы:

```
ssh-keygen -t ed25519 -C "devops-lab"  
ssh-copy-id user@app-server-1  
# при наличии второго сервера:  
ssh-copy-id user@app-server-2
```

Проверьте подключение без пароля:

```
ssh user@app-server-1 "hostname && whoami"
```

Если вы работаете в изолированной лабораторной сети без DNS, добавьте записи хостов в /etc/hosts на control-node, например:
192.168.56.11 app-server-1

Jenkins: автоматическая сборка проекта

Установить Jenkins, настроить его после первого запуска, создать Pipeline-проект и написать Jenkinsfile, который автоматически забирает код из репозитория, собирает и тестирует его

Установка Jenkins на Debian 13

Официальная инструкция: <https://www.jenkins.io/doc/book/installing/linux/>.

Установите Java — на Debian 13 актуален OpenJDK 21:

```
wget https://download.oracle.com/java/21/latest/jdk-21\_linux-x64\_bin.deb
```

```
dpkg -i jdk-21_linux-x64_bin.deb
```

```
java -version
```

Установите Jenkins:

```
cd /tmp
```

```
wget https://get.jenkins.io/debian-stable/jenkins_2.568.2_all.deb
```

```
sudo apt install ./jenkins_2.568.2_all.deb
```

Запустите и включите автозапуск службы:

```
sudo systemctl enable --now jenkins
```

```
sudo systemctl status jenkins
```

По умолчанию Jenkins слушает порт **8080**. Откройте `http://<IP control-node>:8080` в браузере.

Если пакет `jenkins.io-2026.key` недоступен на момент выполнения работы (ключ ежегодно обновляется), проверьте актуальное имя файла ключа на странице <https://pkg.jenkins.io/debian-stable/>

Получите пароль администратора для разблокировки:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Введите его на странице Unlock Jenkins.

На шаге выбора плагинов выберите **Install suggested plugins** (набор плагинов, достаточный для большинства задач CI).

Создайте первого администратора (логин/пароль/e-mail).

Подтвердите Jenkins URL и завершите настройку.

Установка дополнительных плагинов (для Pipeline и работы с Git)

В разделе **Manage Jenkins** → **Plugins** → **Available plugins** убедитесь, что установлены (обычно ставятся по умолчанию в наборе suggested):

- Git
- Pipeline
- Pipeline: Stage View

Подготовка демонстрационного репозитория

Для лабораторной подойдёт любой небольшой проект. Пример — простое Python-приложение с тестами. Создайте на control-node (или в вашем GitHub-аккаунте) репозиторий со следующей структурой:

```
demo-app/
├── app.py
├── test_app.py
├── requirements.txt
└── Jenkinsfile
```

app.py:

```
def add(a, b):
    return a + b
```

```
if __name__ == "__main__":
    print(add(2, 3))
```

test_app.py:

```
from app import add
```

```
def test_add():
    assert add(2, 3) == 5
```

requirements.txt:

```
pytest
```

Опубликуйте репозиторий (например, на GitHub, либо на альтернативном git-хостинге — см. раздел 1.3) — Jenkins будет забирать код из него по URL.

Написание Jenkinsfile

Создайте файл Jenkinsfile в корне репозитория. Используется **декларативный синтаксис Pipeline** (официальная документация синтаксиса: <https://www.jenkins.io/doc/book/pipeline/syntax/>):

```
pipeline {
    agent any

    options {
        timestamps()
    }
}
```

```

}

stages {
    stage('Checkout') {
        steps {
            checkout scm
        }
    }

    stage('Setup') {
        steps {
            sh 'python3 -m venv venv'
            sh './venv/bin/pip install --upgrade pip'
            sh './venv/bin/pip install -r requirements.txt'
        }
    }

    stage('Build') {
        steps {
            sh './venv/bin/python -m py_compile app.py'
        }
    }

    stage('Test') {
        steps {
            sh './venv/bin/pytest --junitxml=result.xml'
        }
    }
}

post {
    always {
        junit 'result.xml'
    }
    success {
        echo 'Сборка прошла успешно'
    }
    failure {

```

```
        echo 'Сборка завершилась с ошибкой'
    }
}
}
```

Пояснения:

- agent any — выполнять пайплайн на любом доступном узле Jenkins;
- stages/stage — логические этапы пайплайна, видны в интерфейсе как отдельные блоки;
- steps — конкретные команды на каждом этапе;
- post — действия по завершении (публикация отчёта о тестах, уведомления).

Проверьте, что на control-node установлен python3 и python3-venv:

```
sudo apt install -y python3 python3-venv python3-pip
```

Создание Pipeline-задачи в Jenkins

На главной странице Jenkins нажмите **New Item**.

Введите имя (например, demo-app-pipeline), выберите тип **Pipeline**, нажмите **OK**.

В разделе **Pipeline** выберите **Definition: Pipeline script from SCM**.

SCM: Git, укажите URL вашего репозитория и ветку (main/master).

Путь к Jenkinsfile оставьте по умолчанию (Jenkinsfile в корне).

Сохраните задачу.

Запуск и проверка сборки

Нажмите **Build Now**.

Откройте сборку → **Console Output** — убедитесь, что все стадии (Checkout, Setup, Build, Test) выполнены успешно.

На странице задачи изучите визуализацию стадий (**Stage View**) и результаты тестов (**Test Result**).

Намеренно «сломайте» тест (измените ожидаемое значение в test_app.py), закоммитьте изменение и повторно запустите сборку — убедитесь, что Jenkins корректно отражает падение стадии Test.